Tecnologia Em ADS

Remover registros duplicados

Prof. Dr. Ricardo Luis Balieiro

Análise Exploratória de Dados - EDA

As principais etapas da EDA são:

01. Compreender o problema
02. Explorar o conjunto de dados
03. Tratar de valores ausentes
04. Remover registros duplicados
05. Detectar valores discrepantes - Outliers
06. Estudar as Distribuições de Características
07. Explorar as relações entre as variáveis
08. Preparar funcionalidades
09. Verificar o equilíbrio das classes
10. Dividir o conjunto de dados

Problemas de dados duplicados

Dados duplicados podem criar problemas durante a análise, tais como:

- As vendas podem parecer maiores do que realmente são.
- A contagem de clientes pode se tornar imprecisa.
- As médias e estatísticas podem estar distorcidas.

Perigo Crítico: Data Leakage-Vazamento de Dados

- **Mecanismo da Falha:**
  - Ocorre quando duplicatas estão presentes na base bruta **antes** do split em Treino e Teste (`train_test_split`).
- **Consequência Prática:**
  - A mesma instância acaba presente simultaneamente no conjunto de Treino **e** no conjunto de Teste.
- **Sintoma Técnico:**
  - **Aparentes métricas perfeitas** no teste (Acurácia/AUC infladas → o modelo apenas decorou a amostra de treino que também estava no teste).
  - **Falha catastrófica em produção** → incapacidade de generalização para dados realmente não vistos.

Sempre remover dados duplicados?

Às vezes, as linhas repetidas são válidas, por exemplo:

- compras repetidas pelo mesmo cliente
- múltiplas visitas a um site
- transações recorrentes

Portanto, antes de remover linhas duplicadas, analise se:

- **Essa repetição está realmente incorreta?**

Tipos de Duplicatas: Exatas vs. Parciais

O tipos de duplicatas são:

- **Exatas**
- **Parciais**

Duplicatas exatas

- **Duplicatas Exatas (Fato)** → Linhas onde todas as colunas possuem valores idênticos.

- **Causa comum** → Erro de ingestão, falhas em JOINs de banco de dados ou reenvio de formulários.

ID_Registro	CPF_Cliente	Nome_Cliente	Email	Status_Plano	Data_Atualizacao
101	123.456.789-00	Ana Silva	ana.silva@email.com	Basic	10/01/2026
101	123.456.789-00	Ana Silva	ana.silva@email.com	Basic	10/01/2026
204	987.654.321-11	Carlos Souza	carlos@email.com	Free	01/02/2026
305	987.654.321-11	Carlos Souza	carlos.novo@email.com	Premium	15/03/2026

Duplicatas parciais

- **Duplicatas Parciais (Semânticas)** → Linhas que compartilham a mesma chave de negócio, mas possuem divergência em atributos secundários ou temporais.

- **Causa comum** → Múltiplas interações do mesmo ator ao longo do tempo (ex: compras sequenciais, atualizações cadastrais ou consultas médicas repetidas).

ID_Registro	CPF_Cliente	Nome_Cliente	Email	Status_Plano	Data_Atualizacao
101	123.456.789-00	Ana Silva	ana.silva@email.com	Basic	10/01/2026
101	123.456.789-00	Ana Silva	ana.silva@email.com	Basic	10/01/2026
204	987.654.321-11	Carlos Souza	carlos@email.com	Free	01/02/2026
305	987.654.321-11	Carlos Souza	carlos.novo@email.com	Premium	15/03/2026

Exemplo

- O que acontece se remover a última linha?

ID_Paciente	Status_Exame	Data
P-88	Em Análise	2026-03-01
P-88	Concluído	2026-03-04

Removendo Linhas Duplicadas Exatas

df.duplicated()

- **df.duplicated()** → Método que mapeia e identifica a presença de registros repetidos em um DataFrame.

Retorna uma Series booleana do mesmo tamanho do DataFrame contendo:

- **True** → A linha é considerada uma duplicata (de acordo com as regras definidas).
- **False** → A linha é considerada única/original na verificação.

df.duplicated()

- **Parâmetro `subset` (Onde procurar):**
  - `None` (Padrão) → Considera **todas as colunas** para definir duplicata (busca duplicatas exatas).
  - `['coluna_A', 'coluna_B']` → Considera apenas a **combinação das colunas especificadas** (busca duplicatas parciais/semânticas).

```
# Sintaxe básica do método
mascara_booleana = df.duplicated(subset=['coluna1', 'coluna2'], keep='first')
```

df.duplicated()

Conteúdo do Slide:

- **A Pergunta do Parâmetro** `keep` **:** "Dentro de um grupo de linhas idênticas, **qual ocorrência eu devo MANTER (proteger com** `False` **)** e quais devo marcar como duplicatas (`True`)?"
- **As 3 Opções de Escolha:**
  1. `keep='first'` (Padrão) → Protege a **PRIMEIRA** linha que aparece no arquivo.
  2. `keep='last'` → Protege a **ÚLTIMA** linha que aparece no arquivo.
  3. `keep=False` → **NÃO PROTEGE NENHUMA**. Carimba o grupo inteiro como duplicata.

```
# Sintaxe básica do método
mascara_booleana = df.duplicated(subset=['coluna1', 'coluna2'], keep='first')
```

Parâmetro keep

- **keep=False**

	id_cliente	cpf	nome	status_plano	data_cadastro
1	101	111.222.333-01	Ana Silva	Basic	2026-01-10
3	101	111.222.333-01	Ana Silva	Basic	2026-01-10
10	101	111.222.333-01	Ana Silva	Basic	2026-01-10
0	104	444.555.666-04	Diego Souza	Premium	2026-03-01
7	104	444.555.666-04	Diego Souza	Premium	2026-03-01

- **keep='first'**

	id_cliente	cpf	nome	status_plano	data_cadastro
3	101	111.222.333-01	Ana Silva	Basic	2026-01-10
10	101	111.222.333-01	Ana Silva	Basic	2026-01-10
7	104	444.555.666-04	Diego Souza	Premium	2026-03-01

df.drop_duplicates()

- `subset` **(Definição do Escopo de Remoção):**
  - `None` (Padrão) → Remove a linha apenas se **100% das colunas** forem idênticas (Duplicata Exata).
  - `['coluna_1', 'coluna_2']` → Remove linhas que repetem a **combinação dessas colunas** (Duplicata Parcial).
- `keep` **(Critério de Retenção):**
  - `'first'` (Padrão) → Preserva a **primeira ocorrência** e apaga as demais.
  - `'last'` → Preserva a **última ocorrência** e apaga as anteriores.
  - `False` → **CUIDADO!** Apaga **TODAS** as instâncias do grupo repetido (a entidade desaparece da base).

```
# Sintaxe padrão do método
df_limpo = df.drop_duplicates(subset=['cpf'], keep='last')
```

df.drop_duplicates()

- `inplace` **(Modificação de Memória):**
  - `False` (Padrão) → Retorna uma nova cópia do DataFrame (mais seguro).
  - `True` → Modifica o DataFrame original diretamente na memória.
- `ignore_index` **(Reorganização de Índices):**
  - `False` (Padrão) → Mantém os índices originais (cria "lacunas" na numeração das linhas).
  - `True` → Recria o índice de 0 até $N - 1$ após a deleção das linhas.

```
# Sintaxe padrão do método
df_limpo = df.drop_duplicates(subset=['cpf'], keep='last')
```

Código

```python
import pandas as pd

#=========================================================
#                    L E I T U R A
#=========================================================
# Carregando o Iris Dataset baixado do Kaggle
df = pd.read_csv("Dados/09 - Clientes-Planos.csv")

#=========================================================
#                  INSPEÇÃO GERAL
#=========================================================
# print("\nTamanho do Dataset (Linhas, Colunas):")
# print(df.shape)

# print("\n\nTipo das Variáveis\nDados:")
# print(df.dtypes)

# print("\nSeleção linhas:")
# print(df.head(10))

# print("\nResumo Estatístico (Média, Mín, Máx...):")
# print(df.describe())
```

Código

```
#=========================================================
#                   DUPLICATAS EXATAS
#=========================================================
# Total de duplicatas exatas
print(f"Total de duplicatas exatas: {df.duplicated().sum()}")

# Retorna as linhas que são exatas
df_duplicadas = df[df.duplicated(keep='first')] #'first', 'last', False
# print(f"\n\nLinhas duplicadas:\n {df_duplicados.head(15).sort_values(by="cpf")}")
print(f"\n\nLinhas duplicadas:\n {df_duplicadas.head(15).sort_values(by=["cpf", "id_cliente"])}")


# Remove linhas duplicatas exatas
df_sem_dup_exatas = df.drop_duplicates()
print(f"\n\nLinhas após remover duplicatas exatas: {len(df_sem_dup_exatas)}")
print(f"\nLinhas sem duplicadas exatas:\n {df_sem_dup_exatas.head(15).sort_values(by=["cpf", "id_cliente"])}")
```

Removendo Linhas Duplicadas Parciais

Ordenação Temporal (Pré-requisito Obrigatório)

- **Por que Ordenar Antes de Tratar Duplicatas Parciais?**
  - Em duplicatas parciais (mesmo `cpf`), registros possuem datas diferentes.
  - Se aplicarmos `drop_duplicates` na base desordenada, o Pandas reterá a linha que estiver no topo do arquivo por acaso — e **não a mais recente**.

- Para ordenar por data, primeiro é necessário converte-la para datetime. Pandas geralmente lê as datas como (object ou string).

```
Tipo das colunas/variáveis:
id_cliente          int64
cpf                   str
nome                  str
status_plano          str
data_cadastro         str
dtype: object
```

Convertendo para o tipo datetime

```
# Ajusta o tipo para datetime para ordenação correta
df_sem_dup_exatas['data_cadastro'] = pd.to_datetime(df_sem_dup_exatas['data_cadastro'])
```

Verificando as duplicatas parciais

```
# Ajusta o tipo para datetime para ordenação correta
df_sem_dup_exatas['data_cadastro'] = pd.to_datetime(df_sem_dup_exatas['data_cadastro'])

# Ordenar por CPF e Data de Cadastro
df_ordenado = df_sem_dup_exatas.sort_values(by=['cpf', 'data_cadastro'], ascending=[True, True])

# Visualizar o DataFrame ordenado por CPF e Data Cadastro
print(f"\n\nDados ordenados:\n {df_ordenado.head(15)}")
```

Removendo as duplicatas parciais

```
# Exclui duplicatas parciais mantendo a última informação (mais recente).
# reset_index(drop=True) ==> Recria o índice no df_final
df_final = df_ordenado.drop_duplicates(subset=['cpf'], keep='last').reset_index(drop=True)

# Exibição do resultado consolidado (8 clientes únicos)
print(f"\n\nDados tratados:\n {df_final.head(15)}")
```

F I M

“Nossa história, depositada nas mãos de Deus, pode ser reescrita a qualquer momento.”

— PE. LUÍS ERLIN (9 MESES COM MARIA)

Prof. Dr. Ricardo Luis Balieiro
ricbalieiro@gmail.com